ROS2 Cybersecurity Analysis and Defense

Steven Chen

1. Introduction

Cyber-physical security is a critical requirement for modern robotic systems. A mobile robot such as TurtleBot3 depends on networked communication between controllers, sensors, middleware, and actuators. If an attacker can join the robot communication graph, inject motion commands, alter network traffic, or corrupt perception data, the result can be unsafe physical behavior rather than only a software failure.

This project evaluates ROS2 security risks and defenses using a TurtleBot3/Gazebo simulation environment. The work focuses on the robot command path, especially velocity control through `/cmd_vel`, and studies how unsecured ROS2 communication can be attacked and how SROS2 can reduce those risks.

The original experimental environment used multiple virtual machines. During the project, the environment was migrated to a more maintainable hybrid WSL2 + Docker architecture:

- WSL host: Gazebo and Robot Simulation
- Docker 1: Controller Docker
- Docker 2: Attacker Docker

1.1 Technical Background

ROS2 is the communication framework used by this project. In ROS2, processes are organized as a ROS graph of nodes that communicate through topics, services, actions, and parameters. ROS2 nodes are independent computational processes in the graph [1], and topics provide strongly typed publish/subscribe communication between nodes [2]. This is why command topics such as `/cmd_vel` are security-critical: any node that can publish valid velocity messages into the command path may influence robot motion.

ROS2 communication is implemented through a middleware abstraction called RMW. In this project the selected RMW implementation is `rmw_fastrtps_cpp`, which uses eProsima Fast DDS. ROS2 commonly uses DDS/RTPS middleware for discovery, serialization, and transport. The ROS2 middleware documentation explains the role of different DDS/RTPS vendors in ROS2 communication [3]. This matters for the DoS and MITM experiments because the attacks target the DDS/RTPS communication layer rather than only ROS2 application code.

DDS Security is the security standard used underneath ROS2 security. The OMG DDS Security specification defines security plugin interfaces for authentication, access control, cryptographic operations, logging, and data tagging [4]. These functions map directly to the security goals evaluated in this project: authenticating legitimate participants, restricting topic access, protecting message integrity, and preventing unauthorized payload modification.

SROS2 is the ROS2 tooling and runtime integration that enables DDS Security for ROS2 systems. The ROS2 security documentation explains that ROS2 security uses middleware security plugins and configuration files to enable encryption, authentication, and policy-based access control for ROS graph participants [5]. The SROS2 setup tutorial states that the `sros2` package provides tools for using ROS2 on top of DDS Security [6]. In this project, SROS2 is configured through a keystore, security enclaves, signed permissions, and environment variables such as `ROS_SECURITY_ENABLE`, `ROS_SECURITY_STRATEGY`, and `ROS_SECURITY_KEYSTORE`.

Security enclaves are important to the experimental design. A security enclave defines the identity and permission boundary for one or more ROS2 processes. The ROS2 security documentation describes enclaves as policies that can protect ROS communications for nodes, processes, users, devices, or an entire graph [5]. This project uses separate enclaves such as `/controller`, `/attacker`, `/gazebo`, `/mitm_controller`, and `/mitm_robot` to test whether SROS2 can distinguish legitimate command publishers from unauthorized attackers.

Gazebo is the robot simulation environment used to make the security effects observable as physical robot behavior. ROS2 provides Gazebo integration tutorials for launching robot simulations [7], and this project uses `ros_gz_bridge` to bridge ROS topics and Gazebo Transport topics between the ROS2 command path and the simulated TurtleBot3 [8].

The network MITM experiments use standard layer-2 and Linux networking mechanisms. ARP is the protocol that maps network-layer addresses to Ethernet hardware addresses [9], so ARP cache poisoning can place an attacker on the packet path in a shared layer-2 network. The project implements the ARP poisoner with Scapy packet crafting [10], applies delay and packet loss with Linux `tc netem`

[11], records packet-path evidence with `tcpdump` [12], and isolates the attacker, controller, and robot endpoints with Docker bridge networking [13].

2. Project Goals

The project has four main goals:

1. Reproduce command injection attacks against an unsecured ROS2 TurtleBot3 system.
2. Evaluate SROS2 as an authentication and access-control defense.
3. Test whether SROS2-secured communication can still suffer availability degradation under DDS/RTPS flood traffic.
4. Prepare a network-layer MITM experiment using an isolated Docker bridge topology.
5. Define future work for SLAM and perception-data security.

The project is not only an attack experiment, instead, it is an security analysis project over ROS2 and turtlebot3. The larger objective is to build a repeatable research workflow that can compare insecure and secure robot configurations.

3. System Architecture

3.1 Original Direction

The early project plan used multiple virtual machines:

- Robot/Gazebo machine.
- Controller machine.
- Attacker machine.

This topology is conceptually clean for security experiments, but it is expensive to maintain. Multiple VMs require repeated ROS2 setup, network configuration, GUI support, and dependency management.

3.2 Migrated WSL + Docker Architecture

The migrated architecture is:

```
Docker controller/attacker
  publish /cmd_vel_in
        |
        v
WSL host cmd_vel_relay
  subscribe /cmd_vel_in
  publish /cmd_vel
        |
        v
ros_gz_bridge
        |
        v
Gazebo TurtleBot3
```

The architecture was chosen because Gazebo requires reliable GUI support, while controller and attacker nodes benefit from container isolation. The key engineering change is the host-side relay:

- Containers publish to `/cmd_vel_in`.
- The host relay republishes to `/cmd_vel`.
- Gazebo consumes `/cmd_vel` through the ROS-Gazebo bridge.

The relay was introduced because direct DDS communication from Docker containers to Gazebo was unreliable in the WSL2 environment even when endpoints were discoverable. So I have to build a special layer to transfer the packet and to make sure the controller can successfully control the Gazebo Robot.

3.3 Environment Configuration

The Configuration Environment Variable is:

```
ROS_DOMAIN_ID=0
RMW_IMPLEMENTATION=rmw_fastrtps_cpp
FASTDDS_BUILTIN_TRANSPORTS=UDPv4
ROS_AUTOMATIC_DISCOVERY_RANGE=SUBNET
ROS_LOCALHOST_ONLY=0
```

The final scripts are organized as follows:

- `scripts/setup/`
- `scripts/wsl_docker/`
- `scripts/demo/`
- `experiments/`
- `deployment/wsl_docker/`

To avoid competence and pollution between different experiment's envrionment, the demo scripts run a shared cleanup step before starting. The cleanup script stops old tmux sessions, Docker containers, Gazebo processes, relay processes, and resets the ROS2 daemon.

4. Experiment 1: Open ROS2 `/cmd_vel` Injection

4.1 Purpose

The experiment is to verify an unsecured ROS2 graph allows an unauthorized node to publish robot motion commands. This design follows directly from the ROS2 topic model: command messages are exchanged through publish/subscribe topics [2]. If no authentication or access-control layer is enabled, the experiment tests whether another discovered publisher can influence the same command path.

4.2 Threat Model

The attacker can join the same ROS2 domain as the controller and robot. SROS2 is disabled, so the ROS2 Security keystore, enclave identity, and permission checks are not active [5][6]. The attacker does not need a certificate or permission file.

4.3 Method

The controller publishes forward velocity commands at 10 Hz. The attacker publishes turn commands at 50 Hz. Both publish into the command path through `/cmd_vel_in`, and the host relay forwards the resulting stream to `/cmd_vel`. Gazebo consumes the final command stream through the ROS-Gazebo bridge, which connects ROS topics to Gazebo Transport topics [8].

The experiment can be launched with:

```
./scripts/demo/tmux_three_machine_demo.sh open
```

The tmux demo shows three roles:

- `ROBOT_GAZEBO_WSL_HOST`
- `CONTROLLER_DOCKER`
- `ATTACKER_DOCKER`

4.4 Expected and Observed Result

The expected normal behavior is straight forward movement. When the attacker starts, the robot will show a different running routes because turn commands are injected at a higher frequency.

The experiment result shows that when the robots starts at first, the robots runs through a correct routes and seems normal. But when we start our attacker, since our attacker has a higher frequency speed of commands, the robots can't go striaght and instead, it will replay a turn movement at a points.

4.5 Security Finding

Default ROS2 communication does not prevent another discovered node from publishing to a critical command topic. Without authentication and authorization, the robot command path is open to unauthorized publishers. So a security robots network require a

security configuration and if we use it as a normal ROS2 robots, we can only use without a outer network and only with its own loop-back network.

5. Experiment 2: SROS2 `/cmd_vel` Access-Control Defense

5.1 Purpose

This experiment evaluates whether SROS2 can prevent unauthorized command injection while still allowing legitimate control. The design is based on DDS Security, which provides authentication, access control, and cryptographic protection underneath ROS2 security [4][5].

5.2 Threat Model

The attacker may still run in the same ROS2 domain, but the system uses SROS2 with enforced security. The controller has permission to publish command input. The attacker does not have permission to publish command topics. This directly tests SROS2's policy-based authorization rather than only encryption [5][6].

5.3 Policy Design

The migrated policy file is:

```
config/sros2_wsl_docker_policy.xml
```

The policy defines three enclaves. Enclaves are used because ROS2 security policies are attached to runtime identities and permission boundaries [5]:

Enclave	Role	Permission
`/controller`	legitimate controller	can publish `/cmd_vel_in`
`/gazebo`	Gazebo, bridge, relay	can subscribe `/cmd_vel_in` and publish `/cmd_vel`
`/attacker`	unauthorized attacker	cannot publish `/cmd_vel_in` or `/cmd_vel`

The policy can be validated with:

```
./experiments/02_sros2_cmd_vel_defense/validate_policy.sh
```

5.4 Jazzy SROS2 Issue and Fix

A key migration issue was the runtime enclave selection. The initial configuration used the wrong assumption about `ROS_SECURITY_ENCLAVE`. In this Jazzy/FastDDS setup, the runtime enclave must be selected using:

```
ROS_SECURITY_ENCLAVE_OVERRIDE
```

or:

```
--ros-args --enclave /controller
```

Without this, the runtime searched the keystore root instead of the specific enclave directory and produced:

```
couldn't find all security files
```

After switching to `ROS_SECURITY_ENCLAVE_OVERRIDE` and explicit enclave arguments, the security directory resolution was corrected.

5.5 Run Procedure

Generate or refresh SROS2 artifacts:

```
./scripts/wsl_docker/init_sros2_docker.sh
```

Launch the secure demo:

```
./experiments/02_sros2_cmd_vel_defense/run_demo.sh
```

Collect evidence:

```
./experiments/02_sros2_cmd_vel_defense/collect_evidence.sh
```

5.6 Result Status

The SROS2 experiment has been migrated and validated in the WSL + Docker environment:

- The secure controller can publish movement commands through the `/controller` enclave.
- Gazebo, the bridge, and the relay can receive and forward allowed command traffic through the `/gazebo` enclave.
- The attacker enclave lacks permission to publish command topics. In the secure experiment, the unauthorized attacker cannot create the `/cmd_vel_in` publisher, which is the expected access-control result.

5.7 Security Finding

SROS2 changes the ROS2 graph from an open publication model to an authenticated and permission-controlled model. The important defense is not only encryption, but also authorization: only the controller enclave should be allowed to publish command input.

But we wander know if a SROS2 can defend robots from other kinds of attack but not only some command injection.

6. Experiment 3: SROS2 UDP DoS / Availability Test

6.1 Purpose

The SROS2 access-control experiment shows that an unauthorized attacker can be blocked from publishing robot commands. However, authentication and authorization do not automatically guarantee availability. This experiment tests whether a separate attacker container can degrade the SROS2-secured control path by flooding local DDS/RTPS UDP ports. The reason for targeting this layer is that ROS2 communication is implemented through DDS/RTPS middleware [3], while SROS2 protects identity, permissions, and message security rather than acting as a general network-rate limiter [5].

6.2 Threat Model

The controller and Gazebo/relay use SROS2 in enforcement mode. The attacker does not try to create a valid command publisher. Instead, it sends high-rate RTPS-like UDP packets toward DDS ports in the local WSL + Docker lab. The objective is not to bypass SROS2 permissions, but to test whether malformed or excessive DDS-layer traffic can cause resource pressure or control delay. This separates authorization failure from availability degradation, which is important because DDS Security and SROS2 are not substitutes for network resource controls [4][5].

6.3 Method

The secure baseline is the same as Experiment 2:

```
/controller enclave -> publishes /cmd_vel_in
/gazebo enclave -> relays /cmd_vel_in to /cmd_vel and drives Gazebo
/attacker enclave -> not authorized for command publication
```

For observability, the controller was changed from continuous forward motion to a square-like path. This makes timing delays and stop events easier to see. The relay also includes an input watchdog: if `/cmd_vel_in` is not received for 0.5 seconds, it publishes a zero `/cmd_vel` to stop stale motion.

The DoS attacker was executed inside the attacker Docker container. In the recorded run, it targeted discovered DDS ports in the `7400-7423` range with 45 worker threads:

```
Duration: 25.0s
Payload size: 1200 bytes
Threads per port: 3
Target ports: 7400, 7410-7423
```

Evidence was collected under:

```
logs/experiments/log_driven_validation/20260416_043331/sros2_dos/
```

6.4 Observed Result

The UDP flood executed successfully. The attacker sent:

```
613,326 packets in 26.2 seconds
average 23,386 packets/second
```

During the attack, the attacker container CPU usage rose above 250%, confirming that the flood generated significant local load. The controller did not permanently fail, but the logs show short control-path degradation.

The controller normally publishes at 10 Hz. During the recorded run, two intervals show clear stalls:

```
1776285263.296  published=220
1776285266.325  published=222
```

This is approximately three seconds with only two additional published messages. Under normal 10 Hz operation, about thirty messages would be expected.

A second stall occurred later:

```
1776285298.296  published=542
1776285301.402  published=545
```

The relay observed corresponding command-input interruptions:

```
No /cmd_vel_in for 2.93s; published zero /cmd_vel
No /cmd_vel_in for 2.91s; published zero /cmd_vel
```

This means the attack did not take over robot motion, but it did create short periods where the secure control input stopped arriving and the relay safety watchdog had to stop the robot.

6.5 Level 2 Intensity Comparison

After the initial run, the DoS test was repeated with three stronger intensity levels. The same secure baseline was used, but packet size, attack duration, and thread count were increased.

Run	Threads per DDS port	Payload	Attack time	Packets sent	Average rate	Max attacker CPU	Controller stalls	Max stall	Relay watchdogs	Max watchdog
`level2_low`	6	1200 bytes	35s	916,324	24,958 pkt/s	278.15%	3	3.11s	2	2.94s
`level2_medium`	10	1400 bytes	45s	1,375,872	29,190 pkt/s	308.73%	4	3.52s	2	2.63s
`level2_high`	16	1400 bytes	60s	2,281,704	34,909 pkt/s	394.60%	4	3.88s	4	3.08s

The comparison shows a clear escalation in resource pressure:

- The packet rate increased from about 25k pkt/s to about 35k pkt/s.
- The attacker container CPU increased from about 278% to about 395%.
- The maximum controller publish-loop stall increased from 3.11s to 3.88s.
- The strongest run produced four relay watchdog events, each around three seconds.

The high-intensity run gives the strongest evidence of availability degradation. The robot was not permanently disabled, but the secure control path repeatedly lost command input long enough for the relay watchdog to issue stop commands.

6.6 Interpretation

The result should be described as a partial availability impact:

- The attack did not bypass SROS2 authorization.
- The attack did not permanently disable the controller or Gazebo.
- The attack did cause repeated multi-second command-path interruptions.
- Increasing attack intensity increased packet rate, attacker CPU load, and maximum controller stall duration.
- The relay watchdog reduced physical risk by converting missing command input into an explicit stop command.

This is an important distinction. SROS2 protects command integrity and authorization, but it does not eliminate middleware or network-layer availability risks. Availability needs separate defenses such as rate limiting, network isolation, resource limits, watchdogs, and traffic monitoring.

6.7 Measurement Limitation

The first log-driven validation run did not produce a trajectory CSV because the external trajectory recorder was started without the required SROS2 enclave. Host-side `ros2 topic` commands also could not see secure topics such as `/cmd_vel_in` and `/odom` without matching security context. However, Gazebo-side logs showed that the internal trajectory marker node did receive `/odom` and spawned markers during the run.

The next measurement improvement is to start the trajectory recorder and snapshot commands under the `/gazebo` enclave so that trajectory gaps can be quantified directly from CSV data.

6.8 Security Finding

SROS2 is effective for access control, but it is not a complete availability defense. Even when unauthorized publication is blocked, DDS/RTPS-level traffic pressure can still cause short command-path interruptions. In a cyber-physical robot, even short interruptions matter because stale velocity commands or missed command updates can affect physical behavior.

7. Experiment 4: Network MITM / ARP Spoofing

7.1 Purpose

This experiment studies a stronger network attacker who attempts to position itself between the controller and robot. The experiment is based on ARP behavior: ARP maps IP addresses to Ethernet hardware addresses, so forged ARP replies can redirect traffic through an attacker in a shared layer-2 network [9].

7.2 Difference from Experiment 1

Experiment 1 is application-layer command injection:

```
attacker publishes ROS2 messages
```

Experiment 3 is network-path manipulation:

```
attacker attempts to become the path between controller and robot
```

This distinction matters because ARP spoofing requires separate layer-2 identities. Docker `network_mode: host` is not appropriate for that attack model; Docker bridge networking is used because it gives the controller, robot, and attacker separate container endpoints on the same virtual bridge [13].

7.3 Migrated Docker Bridge Lab

A separate Docker bridge lab was added:

```
deployment/wsl_docker/docker-compose.mitm.yml
```

It creates three endpoints:

Container	Role	IP
`tb3-mitm-controller`	legitimate endpoint	`172.28.0.10`
`tb3-mitm-robot`	target endpoint	`172.28.0.20`
`tb3-mitm-attacker`	MITM endpoint	`172.28.0.50`

The MITM lab uses Docker bridge/NAT networking rather than Docker host networking. This is necessary because ARP poisoning requires separate layer-2 identities [9][13]. The tmux demo is:

```
./experiments/03_network_mitm/run_demo.sh
```

7.4 ARP Helper and Attack Path

A constrained ARP helper was added. It uses Scapy to craft and send ARP packets, rather than relying on an external ARP-spoofing tool [10]:

```
experiments/03_network_mitm/arp_poison_lab.py
```

It is intentionally limited:

- It defaults to dry-run mode.
- It requires `--execute` to send ARP replies.
- It defaults to the isolated `172.28.0.0/24` Docker lab subnet.
- It supports `--restore`.

Dry-run:

```
sudo ./scripts/wsl_docker/mitm_arp_poison.sh --duration 6
```

Execute in the isolated lab:

```
sudo ./scripts/wsl_docker/mitm_arp_poison.sh --execute --restore --duration 20
```

The active MITM path is:

```
controller 172.28.0.10
  -> attacker 172.28.0.50 as poisoned next hop
  -> robot endpoint 172.28.0.20
```

The attacker enables IPv4 forwarding so the connection remains alive, then applies `tc netem` delay/loss to degrade traffic passing through it [11]. This distinguishes a stable MITM from a simple denial of service: the attacker forwards traffic while modifying timing and reliability.

7.5 Open ROS2 MITM Result

The recorded open-mode MITM run generated evidence under:

```
logs/experiments/03_network_mitm/recorded_20260419_120134/
```

Key observations:

Metric	Result
Ping RTT average	547.7 ms
Ping RTT maximum	2383.0 ms
ROS2 command last-latency average	286.3 ms
ROS2 command max-latency observed	2335.0 ms
ROS2 max command gap	1418.9 ms

ARP evidence confirmed that during the attack the controller's ARP table mapped the robot IP to the attacker MAC, and the robot's ARP table mapped the controller IP to the attacker MAC. The attacker also recorded forwarded ICMP and DDS/RTPS UDP traffic with `tcpdump` [12]. This validates that the attacker was positioned in the communication path and could degrade ROS2 command timing.

7.6 SROS2-Protected MITM Result

The SROS2 MITM run generated evidence under:

```
logs/experiments/03_network_mitm/sros2_recorded_20260420_051959/
```

The attack used:

```
tc netem delay 500ms loss 5%
```

During the attack:

```
controller ARP: 172.28.0.20 -> attacker MAC 0a:7e:45:dd:6c:f6
robot ARP:      172.28.0.10 -> attacker MAC 0a:7e:45:dd:6c:f6
ip_forward:     enabled
```

Measured impact:

Metric	Result
Ping RTT average	537.7 ms
Ping RTT maximum	2673.0 ms
SROS2 `/mitm_cmd` last-latency average	264.9 ms
SROS2 `/mitm_cmd` max-latency observed	2174.5 ms
SROS2 max command gap	2531.3 ms

The result supports a layered security conclusion. SROS2 does not prevent ARP poisoning itself, because ARP is below ROS2/DDS Security [9]. However, DDS Security prevents a normal network-path attacker from rewriting protected command payloads into valid commands [4]. The attacker can still delay or drop encrypted DDS/RTPS packets, causing availability degradation.

7.7 SROS2 Gazebo MITM Visual Result

A Gazebo-visible SROS2 MITM variant was added:

```
ATTACK_DELAY_MS=500 ATTACK_LOSS_PERCENT=5 ATTACK_DURATION=45 \
  ./scripts/demo/tmux_sros2_gazebo_mitm_demo.sh
```

This version connects the secure MITM path to Gazebo:

```
secure controller container, enclave /mitm_controller
  -> encrypted/authenticated /mitm_cmd
  -> ARP MITM attacker with delay/loss
  -> secure robot-gateway container, enclave /mitm_robot
  -> unchanged UDP forwarding to WSL Gazebo
  -> secure WSL UDP receiver and cmd_vel relay, enclave /gazebo
  -> Gazebo TurtleBot3
```

The supporting log is:

```
logs/experiments/03_network_mitm/sros2_gazebo_udp_receiver.log
```

This visual run showed:

Metric	Result
UDP receiver samples	175
Average receive rate	8.37 commands/s
Expected normal receive rate	10 commands/s
Maximum ROS command latency	2996.1 ms
Watchdog stop events	13

The Gazebo version intentionally disables command tampering. In SROS2 mode, the correct result is not a clean straight-to-turn modification. The correct result is degraded availability: command delays, short gaps, and watchdog stop commands. A clean command rewrite would require credential compromise or a deliberately misconfigured SROS2 policy, not a normal MITM bypass.

7.8 Security Finding

Network-layer MITM is a different class of risk than publisher injection. SROS2 can protect confidentiality and integrity of DDS messages, but it does not prevent traffic dropping or availability attacks. Network security and ROS2 security should be treated as complementary defenses.

Future Work

The final planned direction is SLAM and perception-data security. Command topics such as `/cmd_vel` control robot motion, but sensor and localization topics control the robot's understanding of the world.

Potential targets include:

- `/scan`
- `/odom`
- `/tf`
- `/map`
- camera topics used by ORB-SLAM3

Future work should test whether manipulated sensor or transform data can cause localization drift, mapping errors, or navigation failures.

9. Contributions

This project contributes:

1. A maintainable WSL + Docker architecture for TurtleBot3 ROS2 security experiments.
2. A verified open ROS2 command-injection experiment.
3. A migrated and validated SROS2 access-control experiment with policy validation and secure startup scripts.
4. A SROS2 availability test showing short command-path interruptions under DDS/RTPS UDP flood traffic.
5. A Docker bridge MITM lab with isolated endpoint identities.
6. A constrained ARP helper for controlled MITM testing.
7. Open and SROS2 MITM evidence runners that record ARP tables, qdisc state, packet captures, and command-latency statistics.
8. Gazebo-visible MITM demos for both open command tampering and SROS2 availability degradation.
9. A presentation/report-oriented experiment structure under `experiments/`.
10. tmux-based demos that show robot, controller, and attacker roles in separate panes.

10. Limitations

The project has several limitations:

- Gazebo is not containerized in the working setup; it runs on the WSL host for reliable GUI support.
- The SROS2 DoS experiment currently has log evidence of command-path interruptions, but the first trajectory CSV run failed because the recorder was not launched with a matching SROS2 enclave.
- The Docker bridge MITM lab now measures ROS2 command latency and Gazebo command-receiver behavior, but the Gazebo visual effect is still less dramatic than open command tampering because SROS2 correctly prevents payload rewriting.
- The SLAM security experiment is documented as future work and has not yet been fully implemented.

11. Conclusion

This project analyzed the security of a ROS2 TurtleBot3 command-control system through a sequence of controlled experiments. The main objective was not simply to migrate an environment, but to evaluate what attacks are possible against ROS2 robot command traffic, what SROS2 can prevent, and where SROS2's protection boundary ends. The WSL + Docker environment was used as the experimental platform because it makes the controller, attacker, robot endpoint, and Gazebo simulation roles repeatable and observable.

The experiments show that an unsecured ROS2 command path is vulnerable to unauthorized publication. In the open ROS2 configuration, an attacker that can join the ROS2 graph can publish competing velocity commands and influence robot motion. This confirms that command topics such as `/cmd_vel` require explicit security controls in cyber-physical robot systems.

The SROS2 access-control experiment shows that DDS Security and enclave-based permissions can effectively block unauthorized command publishers. When the controller enclave is allowed to publish command input and the attacker enclave is not, the attacker cannot create a valid command publisher. This supports the conclusion that SROS2 is effective for authentication, authorization, and command-message integrity.

However, the availability experiments show that SROS2 is not a complete security solution by itself. The DDS/RTPS UDP flood did not bypass SROS2 permissions, but it caused short command-path interruptions and triggered the relay watchdog. Similarly, ARP-based MITM experiments show that SROS2 does not prevent a network-path attacker from delaying or dropping encrypted DDS traffic. In the

SROS2 MITM setting, the attacker could degrade timing and availability, but could not validly rewrite protected command payloads without credential compromise or policy misconfiguration.

Overall, the project demonstrates that ROS2 robot security must be analyzed as a layered problem. SROS2 is important and effective for identity, access control, and message integrity, but it must be combined with network-layer protections, resource controls, monitoring, and safety watchdogs. Future security analysis should extend beyond command topics to perception and localization topics such as `/scan`, `/odom`, `/tf`, maps, and camera streams, because those data channels also affect robot behavior and safety.

References

[1] ROS 2 Documentation, "Nodes", ROS 2 Kilted documentation. Available: https://docs.ros.org/en/kilted/Concepts/Basic/About-Nodes.html

[2] ROS 2 Documentation, "Topics", ROS 2 Kilted documentation. Available: https://docs.ros.org/en/kilted/Concepts/Basic/About-Topics.html

[3] ROS 2 Documentation, "Different ROS 2 middleware vendors", ROS 2 Kilted documentation. Available: https://docs.ros.org/en/ros2_documentation/kilted/Concepts/Intermediate/About-Different-Middleware-Vendors.html

[4] Object Management Group, "DDS-SECURITY - DDS Security", Version 1.2, February 2026. Available: https://www.omg.org/spec/DDS-SECURITY/

[5] ROS 2 Documentation, "ROS 2 Security", ROS 2 Kilted documentation. Available: https://docs.ros.org/en/kilted/Concepts/Intermediate/About-Security.html

[6] ROS 2 Documentation, "Setting up security", ROS 2 Jazzy documentation. Available: https://docs.ros.org/en/jazzy/Tutorials/Advanced/Security/Introducing-ros2-security.html

[7] ROS 2 Documentation, "Setting up a robot simulation, Gazebo", ROS 2 Jazzy documentation. Available: https://docs.ros.org/en/jazzy/Tutorials/Advanced/Simulators/Gazebo/Gazebo.html

[8] ROS 2 Package Documentation, "ros_gz_bridge", Jazzy package documentation. Available: https://docs.ros.org/en/jazzy/p/ros_gz_bridge/

[9] D. C. Plummer, "An Ethernet Address Resolution Protocol", RFC 826. Available: https://www.rfc-editor.org/rfc/rfc826

[10] Scapy Documentation, "Usage". Available: https://scapy.readthedocs.io/en/latest/usage.html

[11] Linux manual page, "tc-netem(8)". Available: https://man7.org/linux/man-pages/man8/tc-netem.8.html

[12] tcpdump/libpcap Documentation, "tcpdump(1)". Available: https://www.tcpdump.org/manpages/tcpdump.1.html

[13] Docker Documentation, "Bridge network driver". Available: https://docs.docker.com/engine/network/drivers/bridge/